JAVA

FROM BEGINNER TO EXPERT

Topics, explanations, runnable code, diagrams, practice



A real learning path

Start with the first program. Build skills through methods, OOP, collections, files, JVM behavior, concurrency, streams, security, and a booking-service capstone.

Read it, run it, change it, explain it.

Java From Beginner to Expert

A structured course for learning Java fundamentals and developing the judgment to design, debug, test, and maintain real programs.

What is inside

Every chapter names its topics, teaches the idea, walks through code, shows the result or reasoning, gives you a practice task, and ends with a recap.

The baseline is stable Java 21+ unless a page labels a feature version-specific. A JDK is required to compile and run examples. An IDE helps, but the early lessons also work from a terminal or JShell.

Examples are original teaching examples arranged around W3Schools, GeeksforGeeks, Oracle Java Tutorials, and current dev.java topic coverage. Technical details are cross-checked against current Java documentation.

Edition notes are dated October 2026. Always follow the release declared by your project.

How to Study with the Code

1. Find the topic

Use the contents page to jump to a named concept. Each chapter starts with its topic list.

2. Predict first

Read the code and write down the output or state change before running it.

3. Run the exact sample

Use the class name as the .java file name. Copy imports and braces as shown.

4. Change one thing

Alter a value, input, boundary, or condition. Explain why output changes.

5. Teach the idea

Use the diagram and glossary to explain the rule, then solve the Level Up task.

Code conventions

Most examples are complete small classes. If a page labels a snippet "method fragment" or "transaction sketch," it is not a standalone Java file; the caption explains the missing context.

Contents and Learning Roadmap

LEVEL 1 • BEGINNER

1. Java, first program, types, operators, input, decisions, loops
2. Methods, arrays, Strings, scope, recursion, debugging

LEVEL 2 • INTERMEDIATE

3. Classes, objects, encapsulation, inheritance, interfaces
4. Collections, generics, exceptions, files, dates

LEVEL 3 • ADVANCED

5. JVM, bytecode, memory, GC, JIT, concurrency
6. Lambdas, streams, tests, date rules, performance

LEVEL 4 • EXPERT

7. Type system, hashing, modules, edge cases, security
8. Architecture, reliability, observability, capstone

Progression: syntax → data → abstraction → runtime → system design. Each later level assumes the earlier ideas and adds new depth.

Prerequisites and Tools

Install

Get a JDK 21 or newer from a trusted distributor. Use IntelliJ IDEA Community, Eclipse, or VS Code with Java support if you want an IDE.

Verify

These commands should print version information. If not, check that the JDK bin folder is on PATH.

```
java --version
javac --version
jshell
```

Compile and run

Save a public class as the same file name, for example Hello.java. Compile with `javac Hello.java`; run with `java Hello`.

Keep a notebook of errors: symptom, cause, fix, and how to prevent a repeat.

A Small Promise

"The most dangerous phrase in the language is, 'We've always done it this way.'"

— Grace Hopper, computer scientist and U.S. Navy rear admiral

Use curiosity as a tool

Learn why an existing rule or design is there. Then use evidence to decide whether a change improves clarity, safety, or speed.

When you get stuck, shrink the problem until one input, one rule, and one output are visible. Then make a prediction and run the experiment.



Java, Your First Program, and Basic Flow

> **Chapter quote**
>
> "Learning never exhausts the mind." — Leonardo da Vinci

Learning objectives

- Compile and run a class
- Use types, operators, input, and control flow
- Trace a program by hand

Topics in this chapter

- What Java is and where it is used
- A short history: Green Project, Oak, Java (1991–1995)
- JDK vs JVM; source, bytecode, compile and run
- main method and console output
- Variables, identifiers, primitive types, String
- Literals, casting, operators, precedence
- Scanner input, if / else, switch
- for, while, do-while, break, continue

Roadmap

Level 1 of 4 • Chapter 1 of 8 • learn → trace code → practise → explain

Core idea and terminology

Java is a general-purpose, strongly typed, class-based language. Its source is compiled to class files that a Java Virtual Machine can run. Java started as Oak within Sun Microsystems' Green Project in 1991 and was publicly introduced as Java in 1995. It matters because a stable language, mature libraries, and JVM implementations support long-lived applications across many operating systems.

A primitive variable stores a value such as an integer or boolean. A reference variable identifies an object such as a String. The compiler checks declared types before execution. `int` is for whole numbers in range; `long` for larger whole values; `double` for approximate fractions; `boolean` for true/false. Use `String.equals` to compare text content.

Key terms

What Java is and where it is used, A short history: Green Project, Oak, Java (1991–1995), JDK vs JVM; source, bytecode, compile and run, main method and console output. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

What Java is and where it is used

Java is a statically typed language that runs on JVM implementations.

A short history: Green Project, Oak, Java (1991–1995)

Oak began in Sun's Green Project in 1991; Java appeared publicly in 1995.

JDK vs JVM; source, bytecode, compile and run

JDK has compiler/tools; JVM loads and executes class files.

main method and console output

The launcher calls the declared main entry point.

Variables, identifiers, primitive types, String

Primitive variables hold values; references identify objects.

Literals, casting, operators, precedence

Operators have precedence; use parentheses when intent could be unclear.

Scanner input, if / else, switch

Scanner reads tokens; validate user input before using it.

for, while, do-while, break, continue

if/switch branch; loops repeat; break and continue alter loop flow.

Example 1 • A complete first program

Save as TipCalculator.java. The public class name matches the file name. `main` is the starting method. The program calculates a tip and prints two values. For real currency calculations use BigDecimal or integer cents; double stores binary approximations.

```
public class TipCalculator {
    public static void main(String[] args) {
        double bill = 48.50;
        double rate = 0.18;
        double tip = bill * rate;
        System.out.println("Tip: $" + tip);
        System.out.println("Total: $" + (bill + tip));
    }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Input and a decision

The remainder operator `%` gives zero for an even integer. The condition in `if` must be boolean. Scanner reads tokens; bad input can raise InputMismatchException, which later chapters teach you to handle.

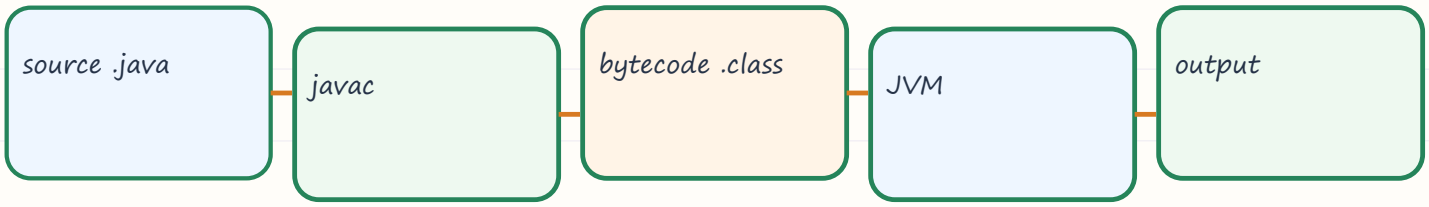
```
import java.util.Scanner;

public class NumberCoach {
    public static void main(String[] args) {
        try (Scanner in = new Scanner(System.in)) {
            System.out.print("Enter a whole number: ");
            int n = in.nextInt();
            if (n % 2 == 0) {
                System.out.println("Even");
            } else {
                System.out.println("Odd");
            }
        }
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

Trace the value of `n = 7`: read `7` → compute `7 % 2 = 1` → condition is false → print `Odd`. Now trace `8`. This is a concrete way to practice prediction before execution.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

Trace the value of `n = 7`: read `7` → compute `7 % 2 = 1` → condition is false → print `Odd`. Now trace `8`. This is a concrete way to practice prediction before execution.

Common mistake

Integer division truncates: `7 / 2` is `3`. `=` assigns, while `==` compares primitive values. Do not compare two Strings with `==`. A loop whose condition never changes may run forever.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Write a parking fee calculator. Read hours, reject negative values, charge a base fee for the first two hours and a per-hour fee after that. Add a weekend branch. Test 0, 2, 3, and an invalid negative input.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
int hours = 3;
if (hours < 0) {
    System.out.println("Hours cannot be negative");
} else {
    int extra = Math.max(0, hours - 2);
    int fee = 5 + extra * 2;
    System.out.println("Fee: " + fee);
}
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Debugging clinic

Integer division truncates: 7 / 2 is 3. `=` assigns, while `==` compares primitive values. Do not compare two Strings with `==`. A loop whose condition never changes may run forever.

Chapter summary and self-check

- Java compiles source to JVM class files.
- Variables have declared types; primitive and reference values behave differently.
- Use if / switch to choose and loops to repeat.
- Integer division and floating-point arithmetic have different rules.
- Predict and test boundary inputs, not just typical ones.

Memory hook

Input → compute → decide → repeat → output.

Five quick checks

- What does javac create?
- What is 7 / 2 as int?
- How compare String contents?
- Which loop checks its condition after the body?
- Why test values on both sides of a threshold?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Methods, Arrays, Strings, and Debugging

Chapter quote

"The first principle is that you must not fool yourself — and you are the easiest person to fool." — Richard Feynman

Learning objectives

- Write methods with contracts
- Work safely with arrays and text
- Debug using stack traces and breakpoints

Topics in this chapter

- Method declaration, parameters, arguments, return
- Overloading, scope, static vs instance methods
- Arrays, indexes, length, enhanced for
- Two-dimensional arrays
- String immutability and StringBuilder
- Command-line arguments and input validation
- Call stack, breakpoints, stack traces
- Basic recursion and termination

Roadmap

Level 1 of 4 • Chapter 2 of 8 • learn → trace code → practise → explain

Core idea and terminology

A method packages a named operation. A parameter is a declared input; an argument is the actual value supplied by the caller. A return type tells callers what comes back. Prefer one clear responsibility per method. Local variables exist only inside their scope. Static methods belong to a class; instance methods work with one object's state.

An array is an object with fixed length and zero-based indexes. `scores.length` is the number of slots, not the last index. Strings are immutable; operations create new String values. Use StringBuilder when assembling many pieces in a loop.

Key terms

Method declaration, parameters, arguments, return, Overloading, scope, static vs instance methods, Arrays, indexes, length, enhanced for, Two-dimensional arrays. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

Method declaration, parameters, arguments, return

Declare a method with access, return type, name, parameters, and body.

Overloading, scope, static vs instance methods

Overloading shares a name with different parameter lists.

Arrays, indexes, length, enhanced for

Arrays have fixed length; indexes begin at zero.

Two-dimensional arrays

Nested arrays can be jagged because each row is an array.

String immutability and StringBuilder

String never mutates; StringBuilder accumulates efficiently.

Command-line arguments and input validation

Scope limits where a local name can be used.

Call stack, breakpoints, stack traces

A recursive call must reach a base case.

Basic recursion and termination

A debugger steps through statements and inspects values.

Example 1 • Extract a method and traverse an ar

The cast happens before division, preserving the fractional average. The enhanced for loop visits every element without managing an index. The empty-array rule is explicit rather than an accidental divide-by-zero result.

```
public class ScoreTools {
    static double average(int[] scores) {
        if (scores.length == 0) {
            throw new IllegalArgumentException("No scores");
        }
        int sum = 0;
        for (int score : scores) sum += score;
        return (double) sum / scores.length;
    }
    public static void main(String[] args) {
        int[] scores = {80, 95, 72};
        System.out.println(average(scores));
    }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Build text without mutating String

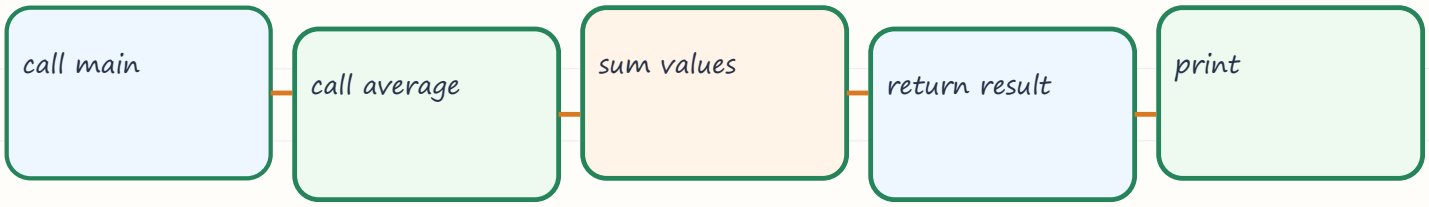
StringBuilder is mutable and useful for repeated appends. `i < items.length` is the safe loop boundary. `i <= items.length` would access one slot past the end.

```
public class Receipt {
    static String format(String[] items) {
        StringBuilder out = new StringBuilder("Items:");
        for (int i = 0; i < items.length; i++) {
            out.append("\n").append(i + 1)
               .append(". ").append(items[i]);
        }
        return out.toString();
    }
    public static void main(String[] args) {
        System.out.println(format(new String[]{"Tea", "Bread"}));
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

For scores [80, 95, 72], trace sum = 247, length = 3, cast to double, divide to get 82.333…. If the result is wrong, inspect each intermediate value instead of rewriting the whole method.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

For scores [80, 95, 72], trace sum = 247, length = 3, cast to double, divide to get 82.333…. If the result is wrong, inspect each intermediate value instead of rewriting the whole method.

Common mistake

Array indexes range from 0 through length - 1. An ArrayIndexOutOfBoundsException points to a bad index; the stack trace shows the call path. A recursive method must move toward a base case or it eventually overflows the stack.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Build a grade report from an int array: minimum, maximum, average, and count above a chosen pass mark. Define what an empty array means. Add a 2D array for a 3-day, 4-meal food log.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
static int countAtLeast(int[] values, int mark) {
    int count = 0;
    for (int value : values) {
        if (value >= mark) count++;
    }
    return count;
}
// Test: [45, 70, 90], mark 70 => 2
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Debugging clinic

Array indexes range from 0 through length - 1. An ArrayIndexOutOfBoundsException points to a bad index; the stack trace shows the call path. A recursive method must move toward a base case or it eventually overflows the stack.

Chapter summary and self-check

- Methods provide names and contracts for behavior.
- Arrays are fixed-size and zero-indexed.
- String is immutable; StringBuilder supports repeated assembly.
- Check the first stack-trace line in your own code and the failing index.
- Every recursive method needs a base case and progress toward it.

Memory hook

Declare → call → trace → return.

Five quick checks

- Parameter vs argument?
- Last valid index for length 8?
- Why cast before dividing for an average?
- When choose StringBuilder?
- What must recursion include?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Classes, Objects, and Object-Oriented Design

Chapter quote

"Simplicity is prerequisite for reliability." — Edsger W. Dijkstra

Learning objectives

- Design classes that protect state
- Use interfaces, composition, and inheritance
- Choose records and enums appropriately

Topics in this chapter

- Classes, objects, fields, constructors, this
- Access modifiers and encapsulation
- Instance vs static members
- Inheritance, override, super, polymorphism
- Abstract classes vs interfaces
- Composition and dependency injection
- Enums, records, packages, imports
- equals, hashCode, toString contracts

Roadmap

Level 2 of 4 • Chapter 3 of 8 • learn → trace code → practise → explain

Core idea and terminology

A class declares a reference type; an object is a runtime instance. A constructor establishes a valid initial state. Encapsulation means callers use a public API while private fields protect invariants. `this` refers to the current object. Static members belong to the class, not one instance.

Inheritance models a true subtype relationship. Dynamic dispatch selects the overridden instance method based on the object's runtime class. Interfaces describe capabilities; composition wires objects together and is often easier to change than a deep hierarchy. Records are concise data carriers with generated accessors and value-style methods; contained mutable objects can still mutate.

Key terms

Classes, objects, fields, constructors, this, Access modifiers and encapsulation, Instance vs static members, Inheritance, override, super, polymorphism. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

Classes, objects, fields, constructors, this

A class is a type declaration; each new call creates an object.

Access modifiers and encapsulation

Constructors establish valid state; this names the current receiver.

Instance vs static members

private protects representation; public methods form the API.

Inheritance, override, super, polymorphism

An override supplies subtype behavior through a supertype reference.

Abstract classes vs interfaces

Polymorphism dispatches an instance call by runtime class.

Composition and dependency injection

An interface defines a capability; an abstract class can share state.

Enums, records, packages, imports

Composition wires collaborating objects together.

equals, hashCode, toString contracts

Records and enums model data and fixed sets of named values.

Example 1 • Encapsulation protects an invariant

The balance cannot be assigned directly by callers. Deposits require a positive amount. Storing cents as an integer avoids floating-point rounding in this small money example; a real system must also handle currency and overflow policy.

```
class BankAccount {
    private final String owner;
    private int cents;
    BankAccount(String owner, int openingCents) {
        if (openingCents < 0) throw new IllegalArgumentException();
        this.owner = owner;
        this.cents = openingCents;
    }
    void deposit(int amountCents) {
        if (amountCents <= 0) throw new IllegalArgumentException();
        cents += amountCents;
    }
    int balanceCents() { return cents; }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Program to a capability

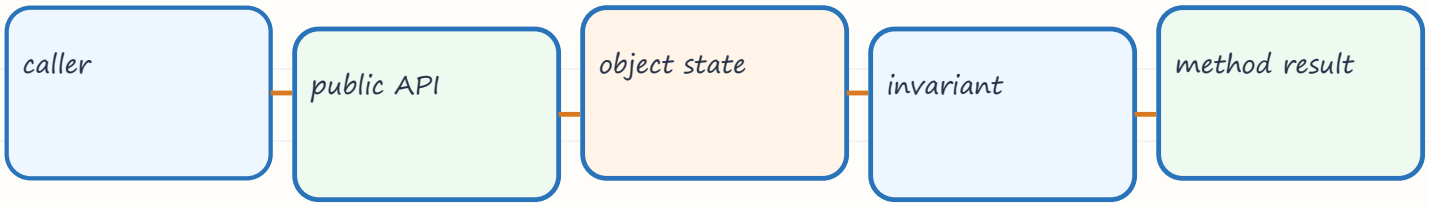
Reminder depends on the Notifier contract rather than a specific email implementation. A test can pass a recording fake; production can pass another adapter. This is a useful seam, not a reason to create an interface for every class.

```
interface Notifier { void send(String text); }
class EmailNotifier implements Notifier {
    public void send(String text) {
        System.out.println("Email: " + text);
    }
}
class Reminder {
    private final Notifier notifier;
    Reminder(Notifier notifier) { this.notifier = notifier; }
    void due(String task) { notifier.send(task + " is due"); }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

Change an account through deposit. Check the invariant before and after each call. Then try to deposit -100: the object rejects the invalid transition rather than storing nonsense.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

Change an account through deposit. Check the invariant before and after each call. Then try to deposit -100: the object rejects the invalid transition rather than storing nonsense.

Common mistake

Do not expose mutable internals. Do not override equals without a compatible hashCode. Use @Override so the compiler catches a signature mistake. Inheritance is not a shortcut for sharing unrelated code.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Model a hotel room with room number, nightly rate, and availability. Add a BookingService that depends on a RoomRepository interface. State which object owns the "no overlapping booking" rule.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
record Guest(String name, String email) { }
interface RoomRepository {
    boolean isAvailable(String roomId, LocalDate from, LocalDate to);
}
// The service asks the repository, then applies
// the booking policy before creating a reservation.
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

Liskov substitution: code using a supertype should still work when given a subtype without surprising broken promises. Favor immutable fields and constructor validation. A record's generated equality compares components; use a class where identity or controlled transitions matter.

Trade-off

Do not expose mutable internals. Do not override equals without a compatible hashCode. Use @Override so the compiler catches a signature mistake. Inheritance is not a shortcut for sharing unrelated code.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- Objects combine state and operations.
- Constructors and methods can protect invariants.
- Interfaces define contracts; composition assembles behavior.
- Override methods carefully and preserve supertype promises.
- Choose records for transparent data, classes for lifecycle and identity.

Memory hook

Hide state; expose actions that keep it valid.

Five quick checks

- What is an object?
- Why make a field private?
- When use an interface?
- What is dynamic dispatch?
- Why must equals and hashCode agree?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Collections, Generics, Exceptions, and Files

Chapter quote

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise." — Edsger W. Dijkstra

Learning objectives

- Choose collection types by behavior
- Use generics and handle failures
- Read and write files with correct lifetimes

Topics in this chapter

- List, Set, Queue, Map and complexity
- ArrayList vs LinkedList trade-offs
- Generics, bounds, wildcards, PECS
- Comparable, Comparator, sorting
- Exception hierarchy, throw, throws, finally
- Try-with-resources and suppressed exceptions
- Path, Files, buffered I/O, charset
- LocalDate, DateTimeFormatter, regex, Optional

Roadmap

Level 2 of 4 • Chapter 4 of 8 • learn → trace code → practise → explain

Core idea and terminology

The Collections Framework offers interfaces with different contracts. List keeps order and duplicates; Set represents uniqueness; Queue models processing order; Map associates keys with values. ArrayList is a strong default for indexed access and append; LinkedList does not make arbitrary indexed access fast.

Generics give compile-time type safety. `List<String>` prevents adding an Integer. Wildcards express variance: producer extends, consumer super. Checked exceptions require handling or declaration; unchecked exceptions often indicate invalid arguments or programming mistakes. Close resources with try-with-resources.

Key terms

List, Set, Queue, Map and complexity, ArrayList vs LinkedList trade-offs, Generics, bounds, wildcards, PECS, Comparable, Comparator, sorting. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

List, Set, Queue, Map and complexity

List preserves sequence and duplicates; Set models uniqueness.

ArrayList vs LinkedList trade-offs

Queue represents work order; Map associates key with value.

Generics, bounds, wildcards, PECS

ArrayList is a good default for indexed list access.

Comparable, Comparator, sorting

Generics make the element type explicit at compile time.

Exception hierarchy, throw, throws, finally

PECS: producer extends, consumer super.

Try-with-resources and suppressed exceptions

Catch specific failures; propagate or translate with context.

Path, Files, buffered I/O, charset

Files and Path APIs support UTF-8 and try-with-resources.

LocalDate, DateTimeFormatter, regex, Optional

java.time separates calendar dates, local times, and instants.

Example 1 • Count values with a Map

Map.merge inserts the first count and combines later values. HashMap offers expected constant-time lookup under ordinary hashing assumptions; it does not promise iteration order. Use LinkedHashMap if encounter order matters.

```
import java.util.*;
public class WordCount {
    public static void main(String[] args) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : List.of("tea", "cake", "tea")) {
            counts.merge(word, 1, Integer::sum);
        }
        System.out.println(counts);
    }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Read a UTF-8 text file safely

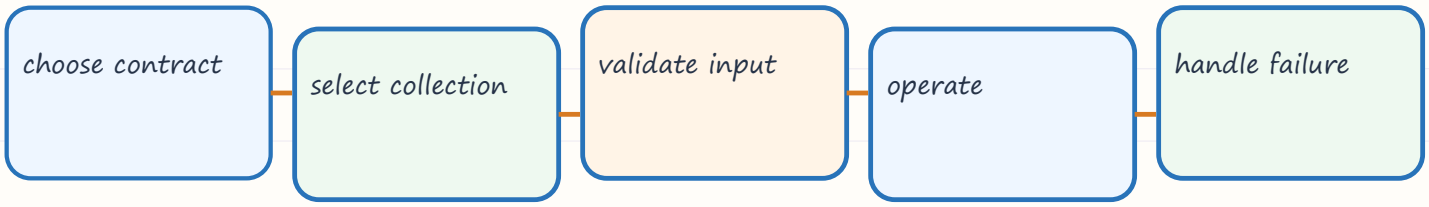
Files.lines returns a lazy stream backed by a resource. Try-with-resources closes it. The default overload uses UTF-8. IOException is declared so a caller must choose how to report or recover from the failure.

```
import java.nio.file.*;
import java.io.IOException;
public class ReadLines {
    static long count(Path path) throws IOException {
        try (var lines = Files.lines(path)) {
            return lines.filter(s -> !s.isBlank()).count();
        }
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

For words [tea, cake, tea], update map: tea→1, cake→1, tea→2. Ask whether capitalization and punctuation should count as differences; normalization is a domain decision.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

For words [tea, cake, tea], update map: tea→1, cake→1, tea→2. Ask whether capitalization and punctuation should count as differences; normalization is a domain decision.

Common mistake

Do not catch Exception and ignore it. Preserve the cause when translating an exception. Do not assume HashMap order. `? extends T` is a producer; you generally cannot add a concrete T to it. `? super T` accepts T values.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Implement a small inventory using `Map<String, Integer>`. Parse a UTF-8 file of `sku,quantity` rows, report malformed line numbers, reject negative quantities, and sort output by SKU.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
Comparator<Map.Entry<String,Integer>> bySku =
    Map.Entry.comparingByKey();
counts.entrySet().stream()
    .sorted(bySku)
    .forEach(e -> System.out.println(
        e.getKey() + ": " + e.getValue()));
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

Use BigDecimal for decimal monetary arithmetic with an explicit scale and rounding mode. Use java.time types such as LocalDate for calendar dates and Instant for timeline points. A local date-time has no time zone. Optional is best for a meaningful possibly-absent return, not every field.

Trade-off

Do not catch Exception and ignore it. Preserve the cause when translating an exception. Do not assume HashMap order. `? extends T` is a producer; you generally cannot add a concrete T to it. `? super T` accepts T values.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- Pick collections by ordering, uniqueness, and lookup needs.
- Generics move type errors to compile time.
- PECS: producer extends, consumer super.
- Try-with-resources closes resources reliably.
- Validate files and time data at the boundary.

Memory hook

List order • Set uniqueness • Map lookup • Queue work.

Five quick checks

- Which collection maps keys to values?
- When use LinkedHashMap?
- What does `? super T` allow?
- Why declare IOException?
- Does LocalDate contain a time zone?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

JVM, Memory, and Concurrency

Chapter quote

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil." — Donald E. Knuth

Learning objectives

- Explain class execution and memory
- Reason about visibility and atomicity
- Use executors and virtual threads carefully

Topics in this chapter

- JDK, JVM, bytecode and class-file versions
- Loading, verification, linking, initialization
- Stack frames, heap objects, references
- Reachability, garbage collectors, JIT compilation
- Java Memory Model and happens-before
- Race conditions, locks, volatile, atomics
- ExecutorService, futures, cancellation
- Virtual threads and structured task lifetimes

Roadmap

Level 3 of 4 • Chapter 5 of 8 • learn → trace code → practise → explain

Core idea and terminology

The JVM is an abstract machine, not a synonym for one implementation. A class loader finds class bytes; verification checks structural safety; linking prepares references; initialization runs static initializers. Method calls use stack frames; objects generally live on the heap. JIT compilers optimize hot code based on runtime evidence. Garbage collection reclaims unreachable heap objects, not files or sockets.

The Java Memory Model defines which writes another thread may observe. Synchronization, volatile access, thread start/join, and concurrent utilities establish happens-before relationships. Atomicity, visibility, and ordering are separate properties.

Key terms

JDK, JVM, bytecode and class-file versions, Loading, verification, linking, initialization, Stack frames, heap objects, references, Reachability, garbage collectors, JIT compilation. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

JDK, JVM, bytecode and class-file versions

Class files contain bytecode and metadata.

Loading, verification, linking, initialization

Loading finds bytes; verification/linking prepare a class.

Stack frames, heap objects, references

Stack frames represent active method calls; heap holds objects.

Reachability, garbage collectors, JIT compilation

GC reclaims unreachable objects, not external resources.

Java Memory Model and happens-before

JIT compilation optimizes frequently executed paths at runtime.

Race conditions, locks, volatile, atomics

Happens-before orders and publishes writes across threads.

ExecutorService, futures, cancellation

Locks/atomics protect invariants; volatile alone is not a compound lock.

Virtual threads and structured task lifetimes

Virtual threads scale blocking tasks, not CPU capacity.

Example 1 • A compound update needs atomicity

A plain `count++` is read, add, write: two threads can interleave and lose an update. AtomicInteger exposes an atomic increment. A synchronized block or lock can protect a larger invariant spanning several fields.

```
import java.util.concurrent.atomic.AtomicInteger;
public class SafeCounter {
    private final AtomicInteger value = new AtomicInteger();
    void increment() { value.incrementAndGet(); }
    int get() { return value.get(); }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Many blocking tasks, Java 21+

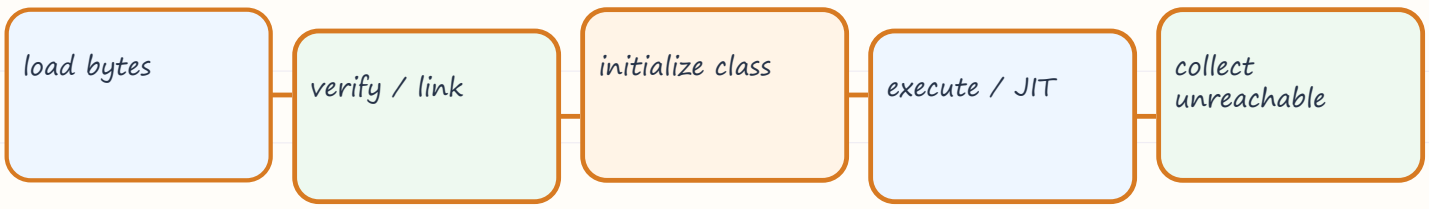
A virtual thread is a Java Thread scheduled by the runtime over a smaller set of carrier threads. This is useful for many blocking tasks. It does not accelerate CPU-bound work or remove limits in a database, network, or remote service.

```
import java.util.concurrent.*;
public class Tasks {
    public static void main(String[] args) throws Exception {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            Future<String> f = executor.submit(() -> {
                Thread.sleep(50);
                return "done";
            });
            System.out.println(f.get());
        }
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

Visibility puzzle: one thread sets `ready=true`; another waits and reads `data`. Without a happens-before edge, seeing ready does not automatically make all earlier writes visible. Use a synchronized protocol, volatile publication, or a concurrency utility.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

Visibility puzzle: one thread sets `ready=true`; another waits and reads `data`. Without a happens-before edge, seeing ready does not automatically make all earlier writes visible. Use a synchronized protocol, volatile publication, or a concurrency utility.

Common mistake

`volatile int count` does not make `count++` atomic. GC timing is not a resource-lifecycle policy. Avoid depending on object layout or a particular collector unless an implementation-specific decision is measured and documented.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Build a service that fetches three independent prices concurrently. Define a timeout, cancellation behavior, partial failure policy, and executor ownership. Make shared state unnecessary where possible.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
try (var executor = Executors
        .newVirtualThreadPerTaskExecutor()) {
    var a = executor.submit(() -> fetch("A"));
    var b = executor.submit(() -> fetch("B"));
    var pa = a.get(2, TimeUnit.SECONDS);
    var pb = b.get(2, TimeUnit.SECONDS);
    System.out.println(pa + pb);
}
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

Virtual threads are permanent since Java 21. Structured concurrency APIs and details are version-sensitive; check the exact target release and preview flags. For CPU parallelism, bound worker count to available processing capacity and measure contention.

Trade-off

`volatile int count` does not make `count++` atomic. GC timing is not a resource-lifecycle policy. Avoid depending on object layout or a particular collector unless an implementation-specific decision is measured and documented.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- The JVM specification defines observable behavior; implementations choose internal strategies.
- Heap reachability drives object reclamation.
- Close external resources explicitly.
- Happens-before gives visibility and ordering guarantees.
- Virtual threads fit blocking concurrency; they are not extra CPU cores.

Memory hook

Ask: shared state? atomic operation? publication edge?

Five quick checks

- Does GC close a socket?
- Why is count++ not atomic?
- What does volatile guarantee?
- When are virtual threads a good fit?
- What bounds CPU parallelism?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Lambdas, Streams, Testing, and Performance

Chapter quote

"Beware of bugs in the above code; I have only proved it correct, not tried it." — Donald E. Knuth

Learning objectives

- Write stream transformations
- Test boundary behavior
- Measure performance with suitable tools

Topics in this chapter

- Functional interfaces and built-in Predicate / Function
- Lambda syntax, method references, capture
- Stream sources, intermediate and terminal operations
- map, filter, flatMap, reduce, collectors
- Optional and date/time APIs
- Unit, integration, property, and concurrency tests
- JMH benchmark design and profiling
- Parallel streams: costs and safe use

Roadmap

Level 3 of 4 • Chapter 6 of 8 • learn → trace code → practise → explain

Core idea and terminology

A functional interface has one abstract method and can be implemented by a lambda. A stream describes a pipeline over data; intermediate operations are lazy, while a terminal operation triggers traversal. Prefer transformations without shared mutable side effects. A for loop may be clearer for stateful logic.

Tests encode examples of expected behavior. Unit tests isolate small rules; integration tests exercise real boundaries. Benchmarks are experiments, not stopwatch guesses: warmup, forks, dead-code elimination, input shape, and environment all matter.

Key terms

Functional interfaces and built-in Predicate / Function, Lambda syntax, method references, capture, Stream sources, intermediate and terminal operations, map, filter, flatMap, reduce, collectors. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

Functional interfaces and built-in Predicate / Function

A functional interface has one abstract method.

Lambda syntax, method references, capture

Lambdas and method references supply behavior as a value.

Stream sources, intermediate and terminal operations

Stream intermediate operations describe lazy transformations.

map, filter, flatMap, reduce, collectors

filter selects, map transforms, flatMap flattens nested streams.

Optional and date/time APIs

Collectors reduce a stream into a list, map, grouping, or summary.

Unit, integration, property, and concurrency tests

Tests capture examples, limits, and regression cases.

JMH benchmark design and profiling

JMH controls warmup and forks for JVM microbenchmarks.

Parallel streams: costs and safe use

Parallel execution adds scheduling, coordination, and contention.

Example 1 • Filter and transform a collection

filter keeps values meeting a predicate. map transforms each remaining element. toList is terminal. Here 105 is allowed because the sample has no 100-point cap; if scores are capped, the business rule needs another operation.

```
import java.util.List;
public class HighScores {
    public static void main(String[] args) {
        List<Integer> scores = List.of(40, 92, 77, 100);
        var result = scores.stream()
            .filter(score -> score >= 70)
            .map(score -> score + 5)
            .toList();
        System.out.println(result); // [97, 82, 105]
    }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Make date rules testable

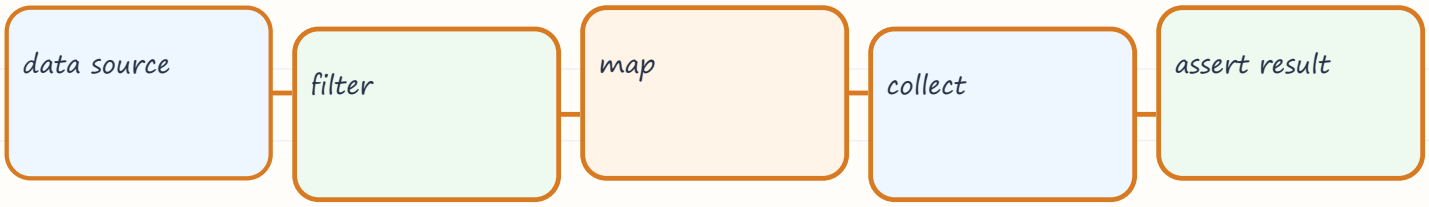
A LocalDate has no time zone. This rule uses a half-open stay interval: check-in is included, check-out excluded. A fixed Clock can make "today" tests deterministic; inject time rather than reading the system clock throughout the domain.

```
import java.time.*;
public class BookingDates {
    static boolean valid(LocalDate from, LocalDate to) {
        return from != null && to != null
            && from.isBefore(to);
    }
    public static void main(String[] args) {
        System.out.println(valid(
            LocalDate.parse("2026-06-01"),
            LocalDate.parse("2026-06-04")));
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

Before coding a discount, agree on negative amounts, threshold equality, currency scale, and rounding. Then write tests for below / at / above threshold. Expected results should be independently calculated.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

Before coding a discount, agree on negative amounts, threshold equality, currency scale, and rounding. Then write tests for below / at / above threshold. Expected results should be independently calculated.

Common mistake

parallelStream is not a magic faster mode. It may contend for resources and shared state. Avoid side effects in stream lambdas. A test suite cannot prove all possible behaviors; combine tests with specification, review, and production signals.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Write tests for a date-range overlap method. Cover touching endpoints, same-day booking, reversed dates, and nulls. Then measure a large search only after profiling identifies it as important.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
static boolean overlaps(LocalDate aStart, LocalDate aEnd,
                        LocalDate bStart, LocalDate bEnd) {
    return aStart.isBefore(bEnd)
        && bStart.isBefore(aEnd);
}
// Half-open ranges: [start, end)
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

JUnit 5 gives common assertions and parameterized tests. Property tests can generate many inputs and check general laws. JMH is designed for JVM microbenchmarks. For service-level speed, inspect realistic traces and latency distributions rather than optimizing a tiny synthetic method.

Trade-off

parallelStream is not a magic faster mode. It may contend for resources and shared state. Avoid side effects in stream lambdas. A test suite cannot prove all possible behaviors; combine tests with specification, review, and production signals.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- Lambdas implement functional interfaces.
- Stream pipelines run at terminal operations.
- Prefer pure transformations or clear loops over hidden side effects.
- Tests make assumptions executable; broader evidence still matters.
- Measure realistic workloads before changing performance-sensitive code.

Memory hook

Specify → test → profile → change → measure again.

Five quick checks

- What triggers a stream?
- What does filter return?
- Why does LocalDate lack a zone?
- Name a common benchmark trap.
- When can parallel streams hurt?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Advanced Java: Type System, Modules, and Securi

Chapter quote

"You must drive for simplicity all the time. If you don't, complexity will nail you." — James Gosling

Learning objectives

- Explain type and collection edge cases
- Design module and API boundaries
- Apply secure coding practices

Topics in this chapter

- Type erasure, bounds, PECS, bridge methods
- equals / hashCode, identity, mutability
- HashMap hashing, collisions, resize, complexity
- Class loaders, modules, service providers
- Annotations, reflection, method handles
- Serialization risks, input validation, path safety
- Unicode, locale, time zones, numeric edge cases
- Stable features vs preview, incubator, draft

Roadmap

Level 4 of 4 • Chapter 7 of 8 • learn → trace code → practise → explain

Core idea and terminology

Generics are mostly implemented by type erasure: the compiler checks type arguments and inserts casts where needed. Bounded wildcards express variance. HashMap uses hash codes and equality to find keys; mutable keys are dangerous because changing hash-relevant state after insertion can make lookup fail. Big-O describes growth, not all real costs.

JPMS modules declare dependencies and exports. Reflection can inspect and invoke members but weakens static guarantees and can be constrained by module boundaries. Never deserialize untrusted native Java object streams. Validate external data, use least privilege, protect secrets, and use prepared SQL parameters.

Key terms

Type erasure, bounds, PECS, bridge methods, equals / hashCode, identity, mutability, HashMap hashing, collisions, resize, complexity, Class loaders, modules, service providers. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

Type erasure, bounds, PECS, bridge methods

Type erasure removes most generic arguments at runtime.

equals / hashCode, identity, mutability

equals/hashCode define value comparison and hash bucket identity.

HashMap hashing, collisions, resize, complexity

HashMap relies on stable keys and distributes hash codes.

Class loaders, modules, service providers

Modules declare requirements and exported packages.

Annotations, reflection, method handles

Reflection inspects types dynamically and can bypass static structure.

Serialization risks, input validation, path safety

Prepared statements bind data separately from SQL syntax.

Unicode, locale, time zones, numeric edge cases

Unicode code points can require two UTF-16 char units.

Stable features vs preview, incubator, draft

Preview and incubator features are opt-in and version-sensitive.

Example 1 • A stable value object

A record supplies value-style equals and hashCode. Its component here is an immutable String, so its hash remains stable. If a key contains mutable state, do not mutate that state while the object is stored in a hash-based collection.

```
record UserId(String value) {
    UserId {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("blank id");
        }
    }
}
// immutable value suitable as a map key
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Keep SQL data separate from SQL syn

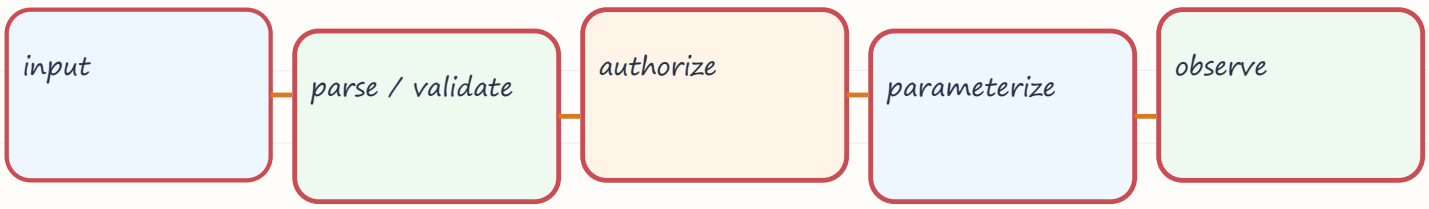
A prepared statement binds email as data rather than concatenating it into SQL. In a real API, use an explicit result type instead of null if absence needs to be distinguished from failure. Transactions and connection ownership also need clear contracts.

```
import java.sql.*;
static User find(Connection c, String email)
        throws SQLException {
    String sql = "SELECT id, email FROM users WHERE email = ?";
    try (PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, email);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next()
                ? new User(rs.getLong("id"), rs.getString("email"))
                : null;
        }
    }
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

A HashMap key must keep equality and hashCode stable while stored. A String key is safe because String is immutable. A mutable key can be inserted in one bucket and later search using a different hash.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

A HashMap key must keep equality and hashCode stable while stored. A String key is safe because String is immutable. A mutable key can be inserted in one bucket and later search using a different hash.

Common mistake

Do not rely on HashMap iteration order. A local date-time without zone cannot represent an unambiguous instant during daylight-saving transitions. Java char is a UTF-16 code unit, not always a full Unicode code point. Validate sizes and ranges before allocating from untrusted input.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Review an API that accepts a file name, user ID, and SQL filter. Mark trust boundaries. Normalize and constrain paths; authorize the user; parameterize values; limit input size; redact secrets in logs.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
Path root = uploadRoot.toRealPath();
Path candidate = root.resolve(fileName).normalize();
if (!candidate.startsWith(root)) {
    throw new SecurityException("outside upload root");
}
// Also consider symlinks and race conditions.
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

As of the JDK 26 documentation, primitive types in patterns, instanceof, and switch are preview. Treat preview and incubating APIs as release-specific, opt-in tools, not permanent contracts. Valhalla, Leyden, and Panama projects explore value layout, startup, and native interop; check project status before adopting drafts.

Trade-off

Do not rely on HashMap iteration order. A local date-time without zone cannot represent an unambiguous instant during daylight-saving transitions. Java char is a UTF-16 code unit, not always a full Unicode code point. Validate sizes and ranges before allocating from untrusted input.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- Generics provide compile-time safety with mostly erased runtime arguments.
- Hash keys require stable equals and hashCode behavior.
- Modules and access boundaries make dependencies explicit.
- Treat I/O, SQL, identity, paths, and secrets as trust boundaries.
- Preview and incubator features can change or be withdrawn.

Memory hook

Boundary first: validate, authorize, constrain, parameterize.

Five quick checks

- What is type erasure?
- Why keep map keys immutable?
- What does PreparedStatement protect?
- Is char always one Unicode character?
- What does preview mean?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Expert Practice and Production Capstone

Chapter quote

"The best way to predict the future is to invent it." — Alan Kay

Learning objectives

- Design a testable service architecture
- Handle concurrency, retries, and operations
- Complete and teach a production-minded capstone

Topics in this chapter

- API contracts, package design, modular monolith
- SOLID principles as heuristics, not rules
- Testing seams: Clock, repository, external gateway
- Observability: structured logs, metrics, traces
- Performance: allocation, contention, latency, profiling
- Reliability: timeouts, retries, idempotency, back pressure
- Concurrency and consistency trade-offs
- Capstone: booking service, review, runbook, teach-back

Roadmap

Level 4 of 4 • Chapter 8 of 8 • learn → trace code → practise → explain

Core idea and terminology

Expert Java work is disciplined judgment. State the invariant first. Put business rules in testable domain code and external effects behind deliberate adapters. Measure latency distributions and resource use. Make failures visible and bounded. Keep the simplest design that satisfies current constraints, with seams where change is plausible.

Capstone: a booking service with room inventory, date ranges, guest counts, cancellation, and a concurrent reservation operation. Define a unique booking key or idempotency key. The core invariant is that two confirmed bookings cannot overlap for the same room. Use a transaction or atomic reservation mechanism once persistence is introduced.

Key terms

API contracts, package design, modular monolith, SOLID principles as heuristics, not rules, Testing seams: Clock, repository, external gateway, Observability: structured logs, metrics, traces. The glossary at the back gives stable short definitions; use these explanations to understand how the ideas work together.

Remember this

Do not memorize code as a spell. Know the input, the rule, the state change, the result, and the failure path.

Topic desk • what each tool does

API contracts, package design, modular monolith

Contracts describe caller-visible behavior and failure.

SOLID principles as heuristics, not rules

A domain invariant states which states are valid.

Testing seams: Clock, repository, external gateway

Adapters isolate storage, clocks, and remote services.

Observability: structured logs, metrics, traces

Metrics count rates and distributions; traces connect a request path.

Performance: allocation, contention, latency, profiling

Profile representative workloads before optimizing.

Reliability: timeouts, retries, idempotency, back pressure

Timeouts bound waiting; retries need idempotency.

Concurrency and consistency trade-offs

Transactions or constraints protect concurrent invariants.

Capstone: booking service, review, runbook, teach-back

A runbook says how to detect, contain, and recover from failure.

Example 1 • Put changing effects behind seams

The service receives its store and clock. Tests can provide in-memory implementations and a fixed instant. Production can use a database and system clock. An abstraction is useful because it protects a decision or enables a test, not merely because interfaces exist.

```
interface ClockSource { Instant now(); }
interface BookingStore {
    Optional<Booking> findOverlap(String room,
        LocalDate from, LocalDate to);
    Booking save(Booking booking);
}
final class BookingService {
    private final BookingStore store;
    private final ClockSource clock;
    BookingService(BookingStore store, ClockSource clock) {
        this.store = store; this.clock = clock;
    }
}
```

Trace it

Write down each variable after its assignment. Predict the output before running this example.

Example 2 • Encode a half-open interval rule

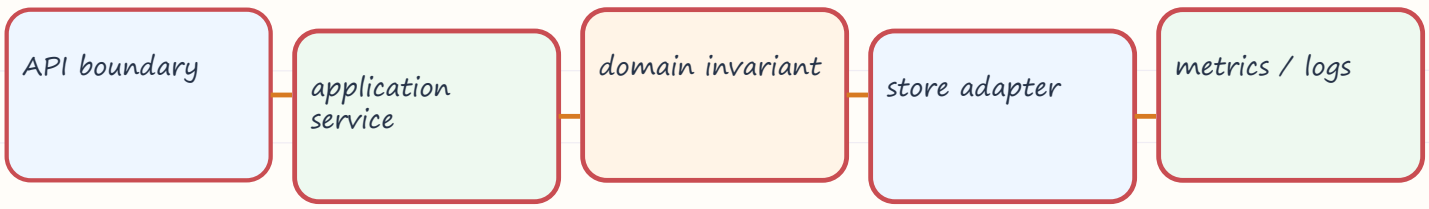
Check-out is excluded, so one guest can leave the same day another arrives. The overlap condition must be enforced atomically at the storage boundary too; two threads can both observe availability before either saves.

```
record Stay(LocalDate checkIn, LocalDate checkOut) {
    Stay {
        if (checkIn == null || checkOut == null
                || !checkIn.isBefore(checkOut)) {
            throw new IllegalArgumentException("invalid stay");
        }
    }
}
static boolean overlaps(Stay a, Stay b) {
    return a.checkIn().isBefore(b.checkOut())
        && b.checkIn().isBefore(a.checkOut());
}
```

Change one thing

Alter one input or rule. State the expected effect first, then run it and explain the difference.

The idea as a picture



Follow the flow from the first box to the last. At each arrow, ask what value or guarantee moves forward.

Think like a debugger

Race walkthrough: two requests check the same room and dates simultaneously. A read-then-write check alone is unsafe. Use a database constraint, serializable transaction, lock, or atomic compare-and-set appropriate to the store. Then test the conflict response and retry behavior.

Byte's hint

A useful diagram tells you where to inspect when the final result is wrong.

Worked case and common traps

Real-life example

Race walkthrough: two requests check the same room and dates simultaneously. A read-then-write check alone is unsafe. Use a database constraint, serializable transaction, lock, or atomic compare-and-set appropriate to the store. Then test the conflict response and retry behavior.

Common mistake

Retries can duplicate effects. Use idempotency keys and persist the outcome. A timeout does not prove the remote operation failed. Time zone rules change; store instants and zone IDs when the business meaning needs them. Do not log tokens or payment details.

Pro tip

Test one normal input, one boundary, and one invalid input. Keep expected behavior explicit.

Your turn • practice lab

Deliver the capstone in five increments: domain + tests; API validation; storage adapter; race/idempotency tests; operational metrics and runbook. Demo success, invalid input, concurrent conflict, and recovery.

- Write down the required behavior before coding.
- Build the smallest version that meets the rule.
- Run the normal case and at least two edge cases.
- Explain why the chosen data structure or method fits.

Did you know?

A deliberately tiny test is often the fastest way to reveal a wrong assumption.

Level Up

Add one requirement that changes a design choice, then state the trade-off.

One worked solution

This is one reasonable starting point, not the only design. Compare your assumptions and edge-case behavior before comparing line count.

```
// Transaction boundary sketch:
BEGIN;
SELECT room FOR UPDATE;
CHECK no overlapping confirmed stay;
INSERT booking with idempotency key;
COMMIT;
// Return prior result for a repeated key.
```

Review questions

Does it satisfy the invariant? What happens for empty, invalid, repeated, or concurrent input? Which part deserves a separate method?

Expert lens • trade-offs and edge cases

Deeper point

Research directions (October 2026): Project Leyden targets startup and time-to-peak performance; Project Valhalla explores value classes and data layout; Project Panama includes the permanent Foreign Function & Memory API, while Vector API work remains incubating. Distinguish shipped JDK features from project prototypes.

Trade-off

Retries can duplicate effects. Use idempotency keys and persist the outcome. A timeout does not prove the remote operation failed. Time zone rules change; store instants and zone IDs when the business meaning needs them. Do not log tokens or payment details.

Pro secret

Measure the behavior you care about. Keep a note of the evidence, the constraint, and the decision that followed.

Chapter summary and self-check

- Design around explicit contracts and invariants.
- Inject time and external systems where tests need control.
- A race must be prevented atomically, not only detected in a prior read.
- Timeout, retry, idempotency, and transaction semantics belong in API design.
- Observability and recovery are part of the feature.

Memory hook

Model → test → constrain → observe → recover.

Five quick checks

- Why inject Clock?
- Why can check-then-save race?
- What does idempotency protect?
- Name a useful production signal.
- Which capstone invariant must never break?

Teach it back

Explain one topic from the opener without reading. Then extend the practice lab with one test that would catch a regression.

Master Cheat Sheet

Purpose	Java form / tool
Whole count	int; use long when range requires
Approximate fraction	double; binary floating point
Exact decimal money	BigDecimal or integer minor units + policy
Ordered growable values	List<T>
Unique membership	Set<T>
Key lookup	Map<K,V>
Work queue	Queue<T> / Deque<T>
Safe resource lifetime	try (resource) { … }
Missing return value	Optional<T> when it clarifies contract
Immutable data carrier	record; shallowly immutable
Concurrent counter	AtomicInteger
Shared invariant	synchronized / Lock / transaction
Many blocking tasks	Virtual threads, JDK 21+
Date without zone	LocalDate
Point on timeline	Instant
Profile JVM microbenchmark	JMH
Inspect a running JVM	JFR, jcmd, profilers; choose by question

Use the choice that fits the contract

Choose by range, ordering, uniqueness, mutation, ownership, and failure behavior—not because a type is fashionable.

Running Glossary • A–M

Argument — Value passed to a method call.
Array — Fixed-length indexed container.
API — Documented surface callers can use.
Autoboxing — Automatic conversion between primitive and wrapper forms.
Bytecode — Instructions in a JVM class file.
Casting — Explicit conversion or type assertion.
Checked exception — Exception the compiler requires callers to catch or declare.
Class — Declaration of a reference type.
Class loader — JVM component that finds and defines classes.
Compiler — Tool that checks and translates Java source.
Constructor — Member that initializes a new instance.
Control flow — Order in which statements execute.
Encapsulation — Restricting representation access to protect rules.
Enum — Type with a fixed set of named constants.
Exception — Object describing an exceptional control-flow condition.
Expression — Code that evaluates to a value.
Field — Variable declared in a class or record.
Functional interface — Interface with one abstract method for lambda targeting.
Garbage collection — Reclaiming unreachable heap objects.
Generic type — Type parameterized by other types.
Heap — Runtime area where objects are generally allocated.
Inheritance — Subtype relationship that can reuse/override behavior.
Instance — One object created from a class.
Interface — Reference type describing a contract.

Running Glossary • N–Z

Idempotency — Repeating a request has one intended effect.
Immutable — State cannot change through the supported API; deep immutability is stronger.
Index — Position used to access an array/list element.
Invariant — Condition that must remain true.
JDK — Development kit with compiler and runtime tools.
JIT compiler — Runtime compiler that optimizes frequently used code.
JLS — Java Language Specification.
JVM — Abstract machine for executing Java class files.
Lambda — Expression implementing a functional interface.
Map — Collection that associates keys and values.
Method — Named operation declared in a class or interface.
Module — Named group of packages with declared dependencies/exports.
Object — Runtime instance with identity and state.
Optional — Container representing a present or absent result.
Package — Namespace that organizes related types.
Parameter — Named input in a method declaration.
Polymorphism — Using a subtype through a supertype contract.
Race condition — Behavior depends on uncontrolled event ordering.
Record — Concise data-carrier class with generated members.
Reference — Value identifying an object or null.
Stream — Lazy or eager pipeline abstraction over elements.
Thread — Path of execution.
Type erasure — Generic arguments mostly removed from runtime representation.
Unchecked exception — Runtime exception not required to be declared/caught.
Virtual thread — Lightweight Java Thread suited to many blocking tasks.
Volatile — Field access rule supporting visibility and ordering.
Wildcard — Unknown/bounded generic argument such as ? extends T.
Zero-based — Indexing begins at 0.

Frequently Asked Questions

Is Java the same as JavaScript?

No. They are separate languages and ecosystems.

Does Java pass objects by reference?

Java passes values; an object variable's copied value refers to the same object.

When use == with objects?

For reference identity; use equals for value/content equality.

Does final make an object immutable?

No. It prevents reassignment of the variable, not mutation of the referenced object.

Does the garbage collector prevent leaks?

No. Reachable objects can accumulate; close external resources explicitly.

Are arrays and ArrayList interchangeable?

No. Arrays have fixed length; ArrayList grows and implements List.

Are checked exceptions always better?

No. Choose exception style to express a meaningful caller obligation.

Do virtual threads make code faster?

Not automatically. They help scale many blocking tasks, not CPU throughput.

Should every loop become a stream?

No. Choose the clearest form for that transformation.

Does compiling prove the program is correct?

No. It checks syntax and type rules, not requirements.

Can two bookings both pass an availability check?

Yes, if concurrent reads race before either write. Enforce the invariant atomically.

Do previews work without flags?

Preview features require explicit release-specific enablement and may change.

Myths vs Facts

Myth: Java is only interpreted.

Fact: JVMs can interpret and JIT-compile; strategies differ.

Myth: GC closes everything.

Fact: It reclaims unreachable heap objects; files need explicit close.

Myth: volatile makes ++ safe.

Fact: Read-modify-write is not atomic.

Myth: HashMap is ordered.

Fact: It makes no iteration-order promise.

Myth: String is mutable.

Fact: String operations return new values.

Myth: a record freezes all nested state.

Fact: Record component references can point to mutable objects.

Myth: parallel is automatically faster.

Fact: Work size, contention, and scheduling determine cost.

Myth: broad catch fixes errors.

Fact: It can hide failures; handle only what you can respond to.

Myth: preview means permanent soon.

Fact: It may change or be withdrawn.

Myth: faster microbenchmark means faster service.

Fact: Production workload and system bottlenecks may differ.

Practice Answers 1–17

1. Which tool compiles source?
Answer: javac.

2. What does a JVM execute?
Answer: Class files containing bytecode.

3. What is 7 / 2 as int?
Answer: 3.

4. How compare String content?
Answer: equals.

5. What does % compute?
Answer: Remainder.

6. First valid array index?
Answer: 0.

7. Length 6: last index?
Answer: 5.

8. Parameter vs argument?
Answer: Declared input vs supplied value.

9. Why cast before average division?
Answer: To perform floating-point division.

10. What is a base case?
Answer: A recursive stopping condition.

11. Why use StringBuilder in a loop?
Answer: To append without creating a chain of intermediate strings.

12. What does private protect?
Answer: Representation and invariants.

13. Class vs object?
Answer: Type declaration vs runtime instance.

14. What is constructor purpose?
Answer: Establish valid initial state.

15. What dispatches an overridden method?
Answer: The runtime class of the receiver.

16. Interface or composition?
Answer: Interface defines contract; composition assembles collaborators.

17. Can record be deeply immutable automatically?
Answer: No.

Practice Answers 18–34

18. Why equals and hashCode together?
Answer: Hash collections need equal keys to have equal hashes.

19. Which collection allows duplicate ordered values?
Answer: List.

20. Which models unique membership?
Answer: Set.

21. Which maps key to value?
Answer: Map.

22. Why use a Queue?
Answer: Represent work ordering.

23. What does a generic type add?
Answer: Compile-time type safety and reusable APIs.

24. PECS expands to?
Answer: Producer Extends, Consumer Super.

25. What guarantees close in try-with-resources?
Answer: AutoCloseable resources close on exit.

26. What does IOException represent?
Answer: A checked I/O failure.

27. Does LocalDate include a zone?
Answer: No.

28. What is a Stream terminal operation?
Answer: An operation that triggers pipeline traversal.

29. What interface does a lambda target?
Answer: A functional interface.

30. What is one test boundary?
Answer: Exactly at a threshold or empty input.

31. What does JMH help with?
Answer: JVM microbenchmarks.

32. What can make parallelStream slower?
Answer: Coordination, small inputs, contention, or blocking.

33. What defines Java language rules?
Answer: The JLS.

34. What does a class loader do?
Answer: Finds and defines class bytes.

Practice Answers 35–50

35. Where are active calls represented?
Answer: Stack frames.

36. When is an object eligible for GC?
Answer: When unreachable from GC roots.

37. What is happens-before?
Answer: A specified ordering/visibility relation between actions.

38. Does volatile make count++ atomic?
Answer: No.

39. What is a race condition?
Answer: An outcome depends on uncontrolled interleaving.

40. What is type erasure?
Answer: Most generic type arguments are removed at runtime.

41. Why avoid mutable map keys?
Answer: Changing equality/hash can make lookups fail.

42. Why prepared SQL statements?
Answer: They bind data separately from query syntax.

43. What is an Instant?
Answer: A point on the UTC timeline.

44. Why use a Clock dependency?
Answer: Control time in tests and domain logic.

45. What does idempotency protect?
Answer: Retries from duplicating intended effects.

46. Why can read-then-write race?
Answer: Two requests can observe the same pre-update state.

47. Name a production metric.
Answer: Latency distribution, error rate, throughput, or saturation.

48. Why not log secrets?
Answer: Logs are broadly copied, stored, and searchable.

49. What is a preview feature?
Answer: A release-specific feature that can change or be withdrawn.

50. What makes an expert decision explainable?
Answer: State the constraint, guarantee, trade-off, evidence, and revisit trigger.

References and 30 / 60 / 90-Day Plan

W3Schools Java Tutorial

w3schools.com/java/ — syntax, variables, control flow, methods, arrays, OOP, exceptions, files, collections, and practice exercises.

GeeksforGeeks Java Tutorial

geeksforgeeks.org/java/java-tutorial/ — basics, methods, OOP, collections, streams, concurrency, memory, practice programs.

Oracle Java Tutorials

docs.oracle.com/javase/tutorial/java/ — foundational language trail. Oracle notes this trail was written for JDK 8.

Current official learning

dev.java/learn/ — updated learning materials for current Java releases.

Java specifications

docs.oracle.com/en/java/javase/26/docs/specs/ — language and virtual-machine specifications.

Project evolution

openjdk.org/projects/leyden/ and openjdk.org/projects/panama/ — project goals and status; drafts are not release guarantees.

Books

Effective Java by Joshua Bloch; Java Concurrency in Practice by Brian Goetz et al.; Head First Java by Kathy Sierra and Bert Bates (check edition/version).

Your 30 / 60 / 90-Day Mastery Plan

Days 1–30 • Core

Finish chapters 1–2. Write 20 small programs; practise types, loops, methods, arrays, and debugging. Explain one topic each week.

Days 31–60 • Structure

Finish chapters 3–4. Build a CLI inventory app using objects, collections, file input, validation, and tests. Ask someone to review the API and error messages.

Days 61–90 • Systems

Finish chapters 5–8. Complete the booking capstone, review concurrency/security, profile a workload, add a runbook, and teach the design.

Weekly rhythm

Learn a concept, build a small example, review one mistake, teach the idea, then rest. Keep a log of what changed in your mental model.

Evidence of mastery

You can explain not only what syntax to type, but what behavior is guaranteed, what can fail, what you measured, and which trade-off you accepted.